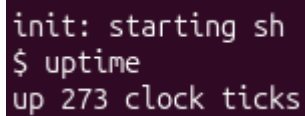


PA 1: Implementing xv6 User Commands and System Calls

Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

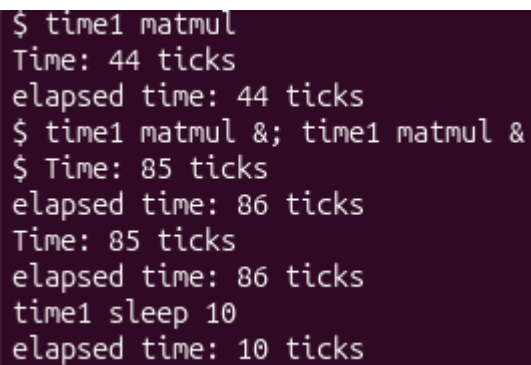
Task 1. (5 pts) Implement the uptime utility

```
$ uptime  
up 273 clock ticks
```



```
init: starting sh  
$ uptime  
up 273 clock ticks
```

Task 2. (10 pts) Implement a time1 command that reports elapsed time.



```
$ time1 matmul  
Time: 44 ticks  
elapsed time: 44 ticks  
$ time1 matmul & time1 matmul &  
$ Time: 85 ticks  
elapsed time: 86 ticks  
Time: 85 ticks  
elapsed time: 86 ticks  
time1 sleep 10  
elapsed time: 10 ticks
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

File

kernel/proc.h

added

```
////////////////////////////////////
int cputime;          // Ticks of CPU time used by process
////////////////////////////////////
```

Where (below)

```
struct inode *cwd;    // Current directory
char name[16];        // Process name (debugging)
```

Why

to count the ticks of the CPU.

Added

What

```
////////////////////////////////////
p->cputime = 0; // Initialize cputime
////////////////////////////////////
```

where (below)

```
found:
p->pid = allocpid();
p->state = USED;
```

Why

Initialize cputime.

File

kernel/trap.c

Added / Change to

```
////////////////////////////////////
// give up the CPU if this is a timer interrupt.
if (which_dev == 2){
    p->cputime++;
    yield();
}
////////////////////////////////////
```

Where

```
// give up the CPU if this is a timer interrupt.
if (which_dev == 2)
    yield();
```

Why add +1 to the counter

Added / Changed

```
////////////////////////////////////
// give up the CPU if this is a timer interrupt.
if (which_dev == 2 && myproc() != 0){
    myproc()->cputime++;
    yield();
}
////////////////////////////////////
```

Where

```
// give up the CPU if this is a timer interrupt.
if (which_dev == 2 && myproc() != 0)
    yield();
```

Why

add +1 to the counter

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

List the files you changed and explain the purpose of each change. Be sure to do git add and git commit as needed and push your changes to your online repo.

Created File with contents

```
kernel/pstat.h
struct rusage {
    uint cputime;
};
```

Why

Is where the actual definition of the struct is held.

File

added

```
////////////////////////////////////
struct rusage;
////////////////////////////////////
```

where (below)

```
struct stat;
```

Why

Declare type of rusage for later

Added

```
/////////  
int wait2(int*, struct rusage*);  
/////////
```

where (below)

```
int wait(int *);
```

Why

Declares the functions and its parameters

File

user/usys.pl

Added

```
#####  
entry("wait2");  
#####
```

where (below)

```
entry("wait");
```

Why

Allows the user to call the new kernel wait2

File

kernel/syscall.h

Added

```
/////////  
#define SYS_wait2 23  
/////////
```

where (below)

```
entry("wait");
```

Why

Every SYS_ needs a specific number, since i created wait2, it needs the next number (23)

File

kernel/syscall.c.

Added

```
/////////  
extern uint64 sys_wait2(void);  
/////////
```

where (below)

```
extern uint64 sys_wait(void);
```

Why

Declares function sys_wait2 exists so it can be referenced.

Added

```
[SYS_wait2] = sys_wait2,  
////////
```

where (below)

```
[SYS_wait] = sys_wait,
```

Why

connect Syscall to the C function that handles them

File

kernel/proc.c

Added

```
////////  
#include "pstat.h"  
////////
```

where (below)

```
#include "proc.h"
```

Why

The upcoming kwait2() method needs a rusage variable. thus the implementation.

Added

Created base on kwait()

```
// added uint64 ruaddr to the call,  
// a 64 unsigned int, represent the rusage address  
// where the caller wants the child's cputime  
int  
kwait2(uint64 addr, uint64 ruaddr)  
<<>>  
////////  
// Temp storage  
struct rusage ru;  
////////  
<<>>  
////////  
// Copies the value from the ZOMBIE child  
ru.cputime = pp->cputime;  
// if address is not 0, and the copy is less than 0  
if (ruaddr != 0 &&  
    // copy data from kernel to user memory.  
    // copies the parent's page table, p->sz is a memory size check  
    // ruaddr is the destiny address in the parents memory  
    // (char *)&ru source of the data in the kernel's memory  
    // sizeof(ru) asks how many bytes to copy.  
    copyout(p->pagetable, p->sz, ruaddr, (char *)&ru,  
            sizeof(ru)) < 0) {
```

```

        // If the copy failed, release both locks.
        release(&pp->lock);
        release(&wait_lock);
        return -1;
    }
}
//////////
<<>>

```

Where (below)

Below method kwait()

Why

Similar to kwait(), uses struct rusage to provide the CPU Time on top of the exit status

File

defs.h

Added

```

//////////
int      kwait2(uint64, uint64);
//////////

```

where (below)

```
int      kwait(uint64);
```

Why

defs.h declares kernel internal functions, so the newly created kwait2 is added.

File

kernel/sysproc.c

Added

```

//////////
uint64
sys_wait2(void)
{
    // holds address of where the exit status should be written
    uint64 p;
    // holds address of where rusage should be written
    uint64 ru;
    // gets first argument from exit status address, and stores it into p
    argaddr(0, &p);
    // gets second argument from rusage address, and stores it into ru
    argaddr(1, &ru);
    // calls the kernel function from proc.c
    return kwait2(p, ru);
}
//////////

```

where (below)

sys_wait(void) method

Why

This is the System call entry point, when wait2() is executed.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

```
xv6 kernel is booting
init: starting sh
$ time matmul
Time: 42 ticks
elapsed time: 42 ticks, cpu time: 42 ticks, 100% CPU
$ time matmul & time matmul &
$ Time: 66 ticks
elapsed time: 67 ticks, cpu time: 69 ticks
s, cpu time: 33 ticks, 49% CPU
elapsed time: 69 ticks, cpu time: 35 ticks, 50% CPU
time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

Getting familiar with the syntax of linux, despite being shorter, task 1 - 3 took similar time to implement to task 4, this because of the lack of familiarity with Linux. After practice, Task 4 took much less time than what would have happened if it was without practice.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes)

The method created only works for one child (no grandchildren) since it is called directly from the time process itself. So it would cause the grandchildren's CPU time to be empty.